

>

💡 WHY DO GRAPHS EXIST?

Arrays and Trees represent linear or hierarchical data. But real-world data is a **network of connections** (social friends, web links, maps, dependencies).

Graphs model entities (**Vertices**) and relationships (**Edges**) with zero restrictions on cycles or connectivity.

💡 THE VISITED LAW

A Tree is an acyclic graph. In Graphs, cycles exist! Traversal **MUST** track `visited[]` or `visited[][]` to prevent infinite recursion loop!

1 GRAPH TERMINOLOGY

- Vertex (V):** Node / entity in network
- Edge (E):** Connection / link between nodes
- Degree:** Count of edges attached to node
- In-Degree / Out-Degree:** Incoming vs outgoing edges
- Path / Cycle:** Sequence of nodes / closed loop

Type	Definition	Key Use Case
Undirected	2-way link (A ↔ B)	Social network friends
Directed	1-way arrow (A → B)	Twitter follower, Course prereq
Weighted	Edge has distance/cost	Google Maps, Flight costs
DAG	Directed Acyclic Graph	Build systems, Topological sort

2 VISUALIZING GRAPH TYPES

UNDIRECTED GRAPH

DIRECTED DAG

graph TD; classDef p fill:#faf5ff,stroke:#7c3aed,stroke-width:2px,color:#6b21a8;font-weight:700; A((0)):::p-->B((1)):::p; A-->C((2)):::p; B-->C

Graph density: Sparse graph ($E \approx V$) vs Dense graph ($E \approx V^2$).

📚 PREREQUISITES & COMPLEXITIES

Concept	Storage / Traversal	Complexity
Adjacency List	<code>List<Integer>[]</code>	Space: $O(V + E)$
Adjacency Matrix	<code>int[V][V]</code>	Space: $O(V^2)$
DFS Traversal	Stack / Call Stack	Time: $O(V + E)$
BFS Traversal	Queue (FIFO)	Time: $O(V + E)$

⚡ THE 4 GRAPH REPRESENTATION TYPES

- Edge List:** `int[][] edges` (raw input in 90% LC problems)
- Adjacency Matrix:** `int[][] grid` (lookup $O(1)$, space $O(V^2)$)
- Adjacency List:** `List<Integer>[]` (optimal for sparse graphs)
- Implicit Grid / Object Nodes:** 2D Matrix OR Node class with `neighbors`

>

1 CONVERSION: EDGE LIST → ADJACENCY LIST

Undirected Graph (2-way insertion):

```
List<Integer>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] e : edges) {
    adj[e[0]].add(e[1]);
    adj[e[1]].add(e[0]); // 2-way link for undirected!
}
```

2 CONVERSION: WEIGHTED EDGES → WEIGHTED ADJ LIST

Edges [u, v, weight] → List<int[]>[]:

```
List<int[]>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] e : edges) {
    int u = e[0], v = e[1], w = e[2];
    adj[u].add(new int[]{v, w});
    // Remove for directed
}
```

>

TYPE 1 GRID GRAPH TRAVERSALS (2D Matrix DFS & BFS) e.g. Number of Islands (LC 200), Max Area of Island (LC 695)

GRID DFS TEMPLATE <pre>void dfs(char[][] grid, int r, int c) { if (r < 0 r ≥ grid.length c < 0 c ≥ grid[0].length grid[r][c] == '0') return; grid[r][c] = '0'; // mark visited for (int[] d : DIRS) dfs(grid, r + d[0], c + d[1]); }</pre>	GRID BFS TEMPLATE <pre>void bfs(char[][] grid, int sr, int sc) { Queue<int[]> q = new LinkedList<>(); q.offer(new int[]{sr, sc}); grid[sr][sc] = '0'; while (!q.isEmpty()) { int[] curr = q.poll(); for (int[] d : DIRS) { int nr = curr[0] + d[0], nc = curr[1] + d[1]; if (nr ≥ 0 && nr < grid.length && nc ≥ 0 && nc < grid[0].length && grid[nr][nc] == '1') { grid[nr][nc] = '0'; // mark when enqueueing! q.offer(new int[]{nr, nc}); } } } }</pre>	KEY RULE Mark <code>grid[nr][nc] = '0'</code> BEFORE adding to queue to avoid TLE/duplicate enqueueing! Time: $O(R \times C)$ Space: $O(R \times C)$ worst case stack/queue
---	--	---

TYPE 2 TREE GRAPH TRAVERSALS (Acyclic Graph DFS & BFS) e.g. Tree Diameter, Graph Valid Tree, Reconstruct Tree

PARENT-TRACK DFS (NO VISITED[] ARRAY NEEDED!) <pre>void dfs(int u, int p, List<Integer>[] adj) { for (int v : adj[u]) { if (v ≠ p) { // Skip immediate parent to avoid backtrack! dfs(v, u, adj); } } }</pre>	TREE BFS LEVEL ORDER <pre>Queue<Integer> q = new LinkedList<>(); q.offer(root); while (!q.isEmpty()) { int size = q.size(); for (int i = 0; i < size; i++) { int u = q.poll(); for (int v : adj[u]) q.offer(v); } }</pre>	AHA INSIGHT In a tree (acyclic graph), checking <code>v ≠ parent</code> replaces full <code>visited[]</code> tracking!
--	---	--

>

TYPE 3 ADJACENCY LIST TRAVERSALS (General Graph DFS & BFS) e.g. Connected Components (LC 323), Path Exists (LC 1971)

GENERAL GRAPH DFS <pre>void dfs(int u, List<Integer>[] adj, boolean[] vis) { vis[u] = true; for (int v : adj[u]) { if (!vis[v]) dfs(v, adj, vis); } }</pre>	GENERAL GRAPH BFS <pre>void bfs(int src, List<Integer>[] adj, boolean[] vis) { Queue<Integer> q = new LinkedList<>(); q.offer(src); vis[src] = true; while (!q.isEmpty()) { int u = q.poll(); for (int v : adj[u]) { if (!vis[v]) { vis[v] = true; q.offer(v); } } } }</pre>	CONNECTED COMPONENTS <pre>int count = 0; for (int i = 0; i < n; i++) { if (!vis[i]) { dfs(i, adj, vis); count++; } }</pre>
--	---	--

>

ADVANCED 1

MULTI-SOURCE BFS (Parallel Propagation)

e.g. Rotting Oranges (LC 994), 01 Matrix (LC 542), Walls and Gates

MULTI-SOURCE CONCEPT

- Enqueue ALL starting sources at t=0
- Process layer-by-layer (minute-by-minute)
- Computes minimum distance to ANY source

💡 AHA

Instead of running BFS from each source separately (slow $O(k \cdot V)$), add all sources to queue initially and run ONE BFS (fast $O(V+E)$)!

TEMPLATE — ROTTING ORANGES (LC 994)

```
int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, time = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == 2) q.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;

    while (!q.isEmpty() && fresh > 0) {
        time++; int size = q.size();
        for (int i = 0; i < size; i++) {
            int[] curr = q.poll();
            for (int[] d : DIRS) {
                int nr = curr[0]+d[0], nc = curr[1]+d[1];
                if (nr >= 0 && nr < grid.length && nc >= 0 && nc < grid[0].length &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; fresh--; q.offer(new int[]{nr, nc});
                }
            }
        }
    }
    return fresh == 0 ? time : -1;
}
```

ADVANCED 2

TOPOLOGICAL SORT (Kahn's In-Degree BFS & Cycle Detection)

e.g. Course Schedule I & II (LC 207, 210), Alien Dictionary

KAHN'S ALGORITHM STEPS

1. Calculate `indegree[]` for all nodes
2. Push nodes with `indegree == 0` to Queue
3. Poll node, decrement neighbor indegrees
4. If neighbor indegree reaches 0 → enqueue
5. If processed count < V → **Cycle exists!**

TEMPLATE — COURSE SCHEDULE II (LC 210)

```
int[] findOrder(int n, int[][] prereqs) {
    List<Integer>[] adj = new ArrayList[n]; int[] in = new int[n];
    for (int i=0; i<n; i++) adj[i] = new ArrayList<>();
    for (int[] p : prereqs) { adj[p[1]].add(p[0]); in[p[0]]++; }

    Queue<Integer> q = new LinkedList<>();
    for (int i=0; i<n; i++) if (in[i] == 0) q.offer(i);

    int[] res = new int[n]; int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); res[idx++] = u;
        for (int v : adj[u]) if (--in[v] == 0) q.offer(v);
    }
    return idx == n ? res : new int[0]; // Cycle if idx != n
}
```

>

DSU

DISJOINT SET UNION (Union-Find with Path Compression & Rank)

e.g. Redundant Connection (LC 684), Number of Provinces (LC 547)

DSU OPERATIONS

- **Find(x)**: Find root + Path Compression
- **Union(x, y)**: Merge sets by Rank
- **Time**: $O(\alpha(N)) \approx O(1)$ amortized

TEMPLATE — DSU CLASS

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int i) {
```

GRAPHS — WHICH ALGORITHM TO USE?

Start: What does the problem ask?

Connected components / island count

→

DFS / visited[]
BFS / [] matrix
DSU tracking

Shortest path in UNWEIGHTED graph

→

BFS (Queue)
Guaranteed shortest path

Shortest path in WEIGHTED graph

→

Dijkstra (PQ)
Min Heap by distance

Parallel multi-source expansion

→

Multi-Source BFS
Enqueue all sources at t=0

Course ordering / prerequisites

→

Topological Sort
Kahn's algorithm in-deg

Cycle detection in undirected graph

→

DSU / DFS
Union returns false if cycle

Connect all points with min cost

→

Kruskal MST
Sort edges + DSU

Grouping dynamic connections

→

Disjoint Set (DSU)
Union-Find with rank

💡 Unweighted Shortest Path = BFS. Weighted Shortest Path = Dijkstra.

🔍 TRIGGER WORDS DIRECTORY

Trigger Word	Algorithm	Key Data Structure
islands, max area, matrix traversal	Grid DFS / BFS	visited[][] / DIRS
shortest path, fewest steps (unweighted)	BFS	Queue<Integer>
shortest path, minimum cost (weighted)	Dijkstra	PriorityQueue<int[]>
rotting, fire spread, distance to 0	Multi-Source BFS	Enqueue all sources
prerequisites, course schedule, alien order	Topo Sort	indegree[] + Queue
redundant edge, graph valid tree	DSU	parent[] + rank[]
connect all points, minimum cost tree	Kruskal MST	Edge sorting + DSU

📁 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why BFS for shortest path, not DFS?	BFS explores level-by-level, ensuring first arrival at target is shortest. DFS can explore deep long paths first.
Why not Dijkstra for negative weights?	Dijkstra assumes distances are non-decreasing. Use Bellman-Ford for negative weights.
Matrix vs Adj List memory?	Adj List is O(V+E) (optimal for sparse). Matrix is O(V²) (wasteful for sparse).

>

EASY — Grid & Basic Graph

LC 200 · Number of Islands

MEDIUM

Pattern: Grid DFS
Key Insight: Sink connected '1's to '0's.

Grid DFS

```
int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') { dfs(grid, r, c); count++; }
    return count;
}

void dfs(char[][] g, int r, int c) {
    if (r < 0 || r ≥ g.length || c < 0 || c ≥ g[0].length || g[r][c] == '0') return;
    g[r][c] = '0';
    for (int[] d : DIRS) dfs(g, r+d[0], c+d[1]);
}
```

LC 695 · Max Area of Island

MEDIUM

Pattern: Grid DFS Area
Key Insight: Return 1 + sum of child DFS calls.

Grid DFS

```
int maxAreaOfIsland(int[][] grid) {
    int max = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == 1) max = Math.max(max, area(grid, r, c));
    return max;
}

int area(int[][] g, int r, int c) {
    if (r < 0 || r ≥ g.length || c < 0 || c ≥ g[0].length || g[r][c] == 0) return 0;
    g[r][c] = 0; int a = 1;
    for (int[] d : DIRS) a += area(g, r+d[0], c+d[1]);
    return a;
}
```

LC 133 · Clone Graph

MEDIUM

Pattern: Object Graph DFS
Key Insight: HashMap tracks original → clone node.

Map Visited

```
Map<Node, Node> map = new HashMap<>();
Node cloneGraph(Node node) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node);
    Node copy = new Node(node.val, new ArrayList<>());
    map.put(node, copy);
    for (Node n : node.neighbors) copy.neighbors.add(cloneGraph(n));
    return copy;
}
```

>

MEDIUM & HARD — DSU & Shortest Path

MEDIUM — Multi-Source & Topo Sort

LC 994 · Rotting Oranges

MEDIUM

Pattern: Multi-Source BFS
Key Insight: Enqueue all 2s initially, step time.

Multi-Source BFS

```
int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, t = 0;
    for (int r=0; r<grid.length; r++)
        for (int c=0; c<grid[0].length; c++)
            if (grid[r][c] == 2) q.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
    while (!q.isEmpty() && fresh > 0) {
        t++; int sz = q.size();
        for (int i=0; i<sz; i++) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int nr=cur[0]+d[0], nc=cur[1]+d[1];
                if (nr ≥ 0 && nr<grid.length && nc ≥ 0 && nc<grid[0].length && grid[nr][nc]==1) {
                    grid[nr][nc]=2; fresh--;
                    q.offer(new int[]{nr, nc});
                }
            }
        }
    }
    return fresh == 0 ? t : -1;
}
```

LC 207 · Course Schedule I

MEDIUM

Pattern: Topological Cycle Detect
Key Insight: Kahn's BFS. Processed count == n.

Kahn's BFS

```
boolean canFinish(int n, int[][] prerequisites) {
    List<Integer>[] adj = new ArrayList[n];
    int[] in = new int[n];
    for (int i=0; i<n; i++) adj[i] = new ArrayList<>();
    for (int[] p : prerequisites) { adj[p[1]].add(p[0]); in[p[0]]++; }
    Queue<Integer> q = new LinkedList<>();
    for (int i=0; i<n; i++) if (in[i] == 0) q.offer(i);
    int count = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); count++;
        for (int v : adj[u]) if (--in[v] == 0) q.offer(v);
    }
    return count == n;
}
```

>

HARD & EXPERT — MST & Alien Order

>

🔍 DRY RUN — Dijkstra Min-Heap Shortest Path

Source K=1, Edges: (1→2, w=1), (1→3, w=4), (2→3, w=1)

Step	PQ Pop (dist, node)	Dist Array	Action / Relaxation
1	(0, 1)	[0, 0, ∞, ∞]	Check neighbors 2 (cost 1) & 3 (cost 4) → Push (1,2), (4,3)
2	(1, 2)	[0, 0, 1, 4]	Check neighbor 3 (1+1=2 < 4) → Relax dist[3]=2! Push (2,3)
3	(2, 3)	[0, 0, 1, 2]	Check neighbors → None. Processed!
4	(4, 3)	[0, 0, 1, 2]	4 > dist[3]=2 → Skip stale entry! ✅

📐 MATHEMATICAL COMPLEXITY DERIVATIONS

Why DFS/BFS is $O(V + E)$:
Every node is enqueued/visited at most once ($O(V)$). Every edge is traversed at most once in directed (twice in undirected) ($O(E)$). Total = $O(V + E)$.

Why Dijkstra is $O((V + E) \log V)$:
Min-heap holds at most E edges. Each edge relaxation does a `pq.offer()` of $O(\log V)$. Total = $O(E \log V + V \log V)$.

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Build Adj List from Edge List in 10s
- ☐ Write Grid DFS with bounds check from memory
- ☐ Write Grid BFS with enqueueing visited mark
- ☐ Write Multi-Source BFS (Rotting Oranges)
- ☐ Code Kahn's Topological Sort BFS from memory
- ☐ Implement DSU with Path Compression
- ☐ Code Dijkstra Shortest Path with PQ
- ☐ Code Kruskal MST with Edge sorting + DSU
- ☐ Identify Graph pattern from problem clues
- ☐ Solve LC 200, 994, 207, 743 within 20m each

📦 GOLDEN RULES — NEVER FORGET

- Rule 1 — Mark Visited on Enqueue**

In BFS, set `visited = true` immediately when pushing to Queue, NOT when polling!
- Rule 2 — Unweighted Shortest Path = BFS**

BFS guarantees shortest path in unweighted graphs. Dijkstra is ONLY for weighted graphs!
- Rule 3 — Skip Stale Dijkstra Entries**

Always check if `(d > dist[u])` continue; when polling from Dijkstra PriorityQueue!